

Home SOC Suite

Pre-Coding Architecture Guide

Decision support document for Python architecture, collector integration, correlation logic, dashboard design, archival integrity, and testing strategy.

Purpose: capture the highest-value design decisions before heavy coding begins, so implementation in PowerShell and Python stays coherent, explainable, and resilient.

1. Current and Assumed End State

Current implemented direction. PowerShell collectors generate endpoint telemetry; analyst scripts summarize and inspect logs.

Assumed final architecture. All collectors and analysts are completed, a Python correlation engine ingests and normalizes telemetry, a dashboard presents live operational status, archived evidence is encrypted and integrity-validated, and a forensic module reconstructs historical activity from archives.

Core pipeline:

- Collect → normalize → correlate → alert → investigate → archive
- Hot path: recent telemetry, detections, dashboard state, and active cases
- Cold path: encrypted archive packages, manifests, and retrospective forensic reconstruction

2. Architecture Principles to Lock In Now

- Do not over-trust telemetry. Every event should carry source and trust metadata.
- Treat missing data as a security signal, not just a reliability bug.
- Separate raw events, normalized events, detections, alerts, and cases.
- Use JSON as raw-event truth and SQLite as operational-query truth.
- Build explainability into every detection from day one.
- Design for multi-timescale correlation: minutes, hours, and days.
- Keep baseline logic versioned and reviewable.
- Make archive integrity a first-class concern, not a final add-on.

3. Canonical Data Model

Define these object types before writing heavy Python logic.

Object	Purpose	Example fields
Raw event	Exactly what a collector	collector_name, raw_payload,

Object	Purpose	Example fields
	observed	observed_at, recorded_at, source_host
Normalized event	Common schema for cross-source correlation	event_type, entity, severity_base, trust_level, parser_confidence
Detection	A rule or scoring match	rule_id, matched_events, score_delta, rationale
Alert	User-visible triage item	alert_id, severity_current, confidence, status, linked_case
Case	Grouped investigative unit	case_id, related_alerts, evidence_refs, analyst_notes, disposition
Evidence artifact	Archive or supporting material	archive_id, manifest_hash, schema_version, chain_prev_hash

Minimum normalized-event fields

- event_id (stable unique identifier)
- schema_version
- collector_name and collector_version
- source_host
- observed_at and ingested_at
- event_type and subtype
- actor / process / destination / file / task identifiers as applicable
- base_severity
- trust_level
- parser_confidence
- raw_event_hash
- ingest_status

4. Trust, Health, and Missing-Data Detection

The system should not only detect malicious activity. It should also detect loss of visibility.

Create a separate class of health and visibility alerts so the SOC can reason about when it may be blind.

Plan these alert families

- Collector silent beyond expected interval
- Event-rate collapse or surge relative to host baseline
- Malformed-event spike
- Timestamp gaps or clock drift anomalies
- File rotation failure or log continuity break
- Cross-collector contradiction (for example, network activity without expected supporting telemetry)

5. Correlation Engine Design

The Python engine should combine deterministic rules and weak-signal scoring.

- Rule layer: clear explainable detections for high-confidence behaviors
- Scoring layer: accumulates smaller unusual signals over time
- Persistent state layer: supports low-and-slow and campaign-style behavior

Recommended correlation windows

- Short window: seconds to minutes
- Operational window: hours
- Campaign window: days to weeks

Important design choice

Do not let current collector output alone define the rules. Define the detections you want, then ensure collectors emit the fields required to support those detections.

6. Baselines, Unknowns, and Rarity

- Version every baseline
- Track baseline age and confidence
- Separate observed behavior from approved-normal behavior
- Model rarity per host, per user, per process, per destination, and per time-of-day

Recommended lifecycle states

- new
- observed
- candidate_normal
- approved_normal
- stale
- revoked

Treat UNKNOWN as a workflow, not a dead-end label. The engine should track first_seen, last_seen, sighting_count, host_count, review_status, and final disposition.

7. Dashboard Requirements

A good dashboard should reduce uncertainty, not hide it.

- collector health and freshness
- ingest lag
- alert confidence
- rule and evidence explanation
- linked raw and normalized events
- open cases and analyst status
- missing-data warnings

Avoid a dashboard that only shows color-coded alerts. It should also show why the system believes what it believes and where visibility is degraded.

8. Archive and Forensic Design

Encryption protects confidentiality; integrity validation protects against silent alteration. The archive layer needs both.

Archive package should include

- encrypted data bundle
- manifest with schema and collector versions
- per-file hashes
- archive creation timestamp and covered time range
- previous-archive hash for chaining
- optional lightweight index for later retrieval

Program 3 should reconstruct narratives, not only search logs. It should answer what happened, in what order, with what confidence, and supported by which evidence artifacts.

9. SQLite and JSON Storage Plan

Layer	Recommended truth source	Why
Collector output	JSON	Preserves original event shape and supports replay/testing
Operational query state	SQLite	Fast filtering, correlation, dashboard reads, and alert history
Archive package	Encrypted raw JSON + manifest	Retains evidence with provenance and integrity metadata
Forensic workspace	Temporary SQLite import	Supports historical

Layer	Recommended truth source	Why
		reconstruction without mutating live state

Avoid dual truth. Raw JSON should remain evidence; SQLite should remain the live operational model.

10. Cross-Collector Validation Opportunities

- Process creation should align with later resource observations
- Outbound network activity should often have nearby DNS unless the connection used a direct IP
- Persistence creation should later align with execution or system startup evidence
- Reboot and wake events should align with collector restart and event continuity
- Security or login context should align with process and network activity timing

11. Testing Strategy to Preserve and Expand

Current practice already in place:

- PowerShell 5.1
- PowerShell 7.5.5
- Standard user context
- Administrator context

That four-way matrix is valuable and should stay part of the project. Extend it to the Python layer and the archive/forensic layer.

Add these test categories before heavy coding

- normal event stream replay
- malformed-event handling
- duplicate and delayed-event handling
- collector silence and health degradation
- baseline poisoning attempt
- low-and-slow multi-day simulation
- archive integrity failure
- schema migration compatibility

12. High-Value Decisions Before Coding

- Freeze the normalized-event schema before building complex rules.
- Decide the exact trust-level model and health-alert thresholds.
- Define alert, case, and evidence-link relationships.

- Document archive manifest structure and key separation strategy.
- Write a compatibility policy for schema and collector versions.
- Define what is in scope and out of scope for tamper resistance.
- Design dashboard widgets around freshness, confidence, and explainability.

13. One-Sentence Risk Summary

Even with all collectors and rules implemented, the most likely remaining weaknesses will be over-trusting telemetry, under-detecting silence, and losing context across time.

14. Build Order Recommendation

- Finalize schema and trust model
- Implement normalization and ingest pipeline
- Implement health and visibility alerts
- Add deterministic rules
- Add scoring and multi-timescale state
- Build dashboard around confidence and freshness
- Implement archive packaging and forensic import

Prepared for architecture planning and Claude-assisted coding decisions.